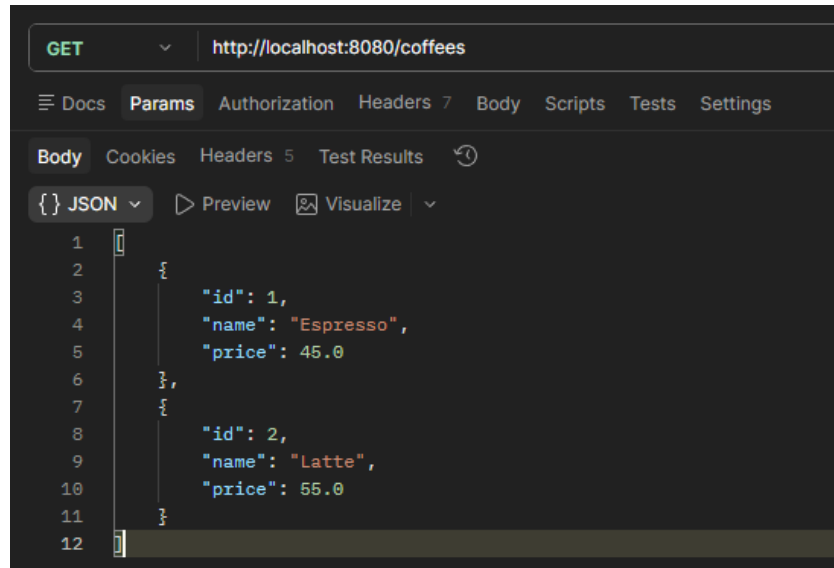


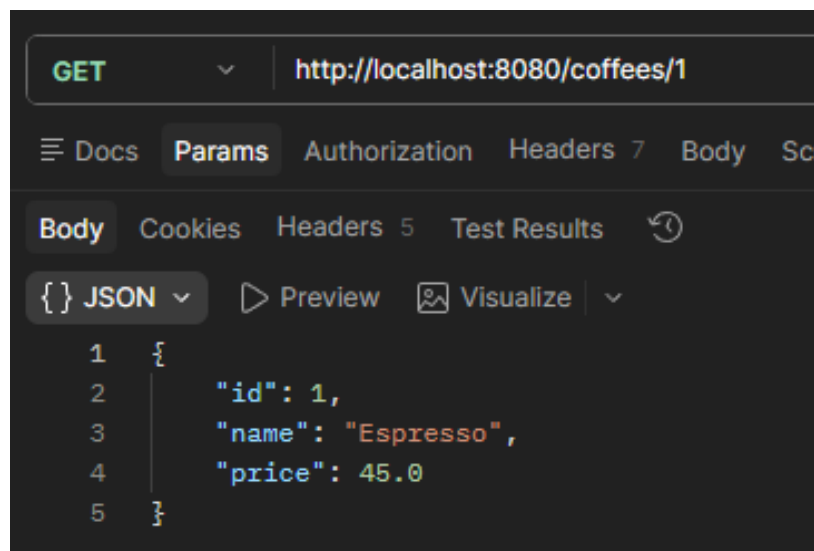
Lab 5 Spring Been Scope

1. ทดลองใช้ HTTPS Method แบบต่าง ๆ

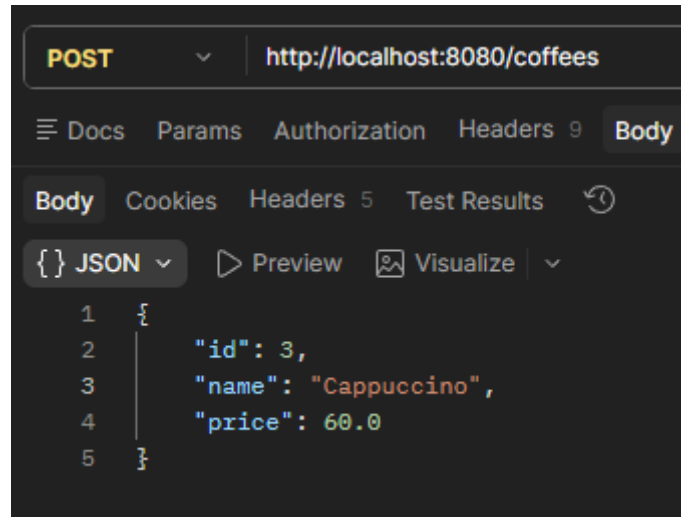
1.1 GET Method แบบดึงข้อมูลทั้งหมด



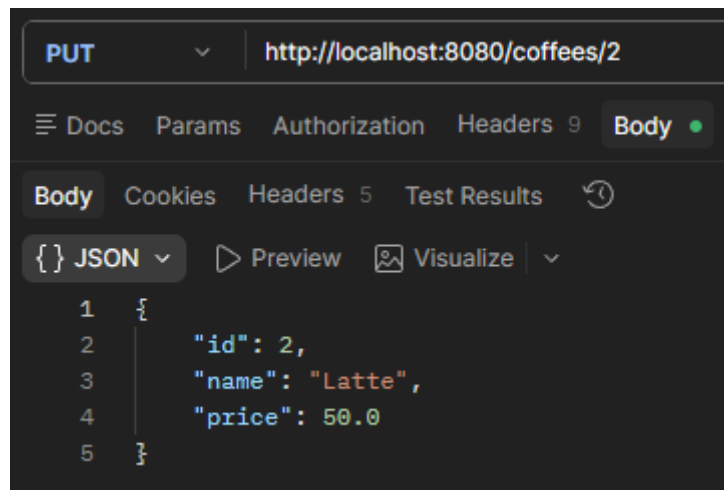
1.2 GET Method แบบดึงเฉพาะ Id ที่ต้องการ



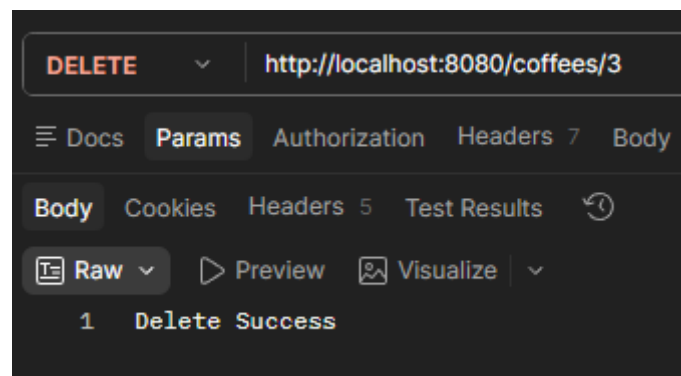
1.3 Post Method เพิ่มข้อมูลตัวที่ 3



1.4 Pull Method เพื่ออัปเดตข้อมูลตามไอดีที่เราต้องการ



1.5 Delete Method ลบข้อมูลที่ไม่ต้องการแล้ว



2. ตอบคำถาม

2.1 HTTP method แต่ละตัว (GET/POST/PUT/DELETE) ต่างกันอย่างไร ยกตัวอย่างจากโปรเจกต์ตัวเอง

ตอบ : GET Method ใช้ดึงข้อมูล , POST Method ใช้เพิ่มข้อมูล , PUT Method ใช้อัปเดตข้อมูล และ DELETE Method ใช้ลบข้อมูล

2.2 ทำไมต้องแยก Controller กับ Service ออกจากกัน มีข้อดีอย่างไรถ้าโปรแกรมโตขึ้น

ตอบ : Controller มีหน้าที่ในการจัดการ Request นั้นๆว่าต้องการทำอะไร แล้วค่อยไปเรียกใช้ Service อีกที ส่วน Service ก็มีหน้าที่ในการจัดการข้อมูลตามคำสั่งที่ได้จาก Controller อีกที โดยการทำแบบนี้มีข้อดีตรงที่ ถ้าโมเดลเราใหญ่ขึ้น เราจะสามารถจัดการโปรแกรมได้ง่าย และสะดวก

2.3 ข้อมูลที่เก็บไว้ใน List ใน memory หายไปตอนไหน และถ้าอยากให้ไม่หายควรทำอย่างไร (ตอบเป็นแนวคิดพอ)

ตอบ : หายไปตอนปิดเครื่อง ปิดเซิร์ฟเวอร์ หรือรันโปรแกรมใหม่ ถ้าไม่อยากให้หายต้องเก็บไว้ใน Database อื่นๆที่ไม่ใช่ใน Ram เช่น MySql , Excel เป็นต้น

2.4 @RestController, @GetMapping, @PostMapping, @PathVariable, @RequestBody แต่ละตัวทำหน้าที่อะไร

ตอบ : @RestController ใช้กำหนดให้คลาสเป็น REST Controller , @GetMapping ใช้กำหนด Method ที่รองรับ GET Method , @PostMapping ใช้กำหนด Method ที่รองรับ POST Method , @PathVariable ใช้รับค่าจาก Request มาแปลงเป็น ข้อมูล เช่น /coffes/1 จะเอา 1 มาเป็น id , @RequestBody ใช้รับค่า JSON ที่ส่งมากับ Request แล้วแปลงเป็น JavaObject

3. ข้อ Bonus

- เพิ่ม endpoint ค้นหาตาม name โดย

Step 1 : เพิ่ม Code ใน CoffeeService.java

```
// Ex Bonus
public List<Coffee> searchByName(String name) {
    List<Coffee> result = new ArrayList<>();

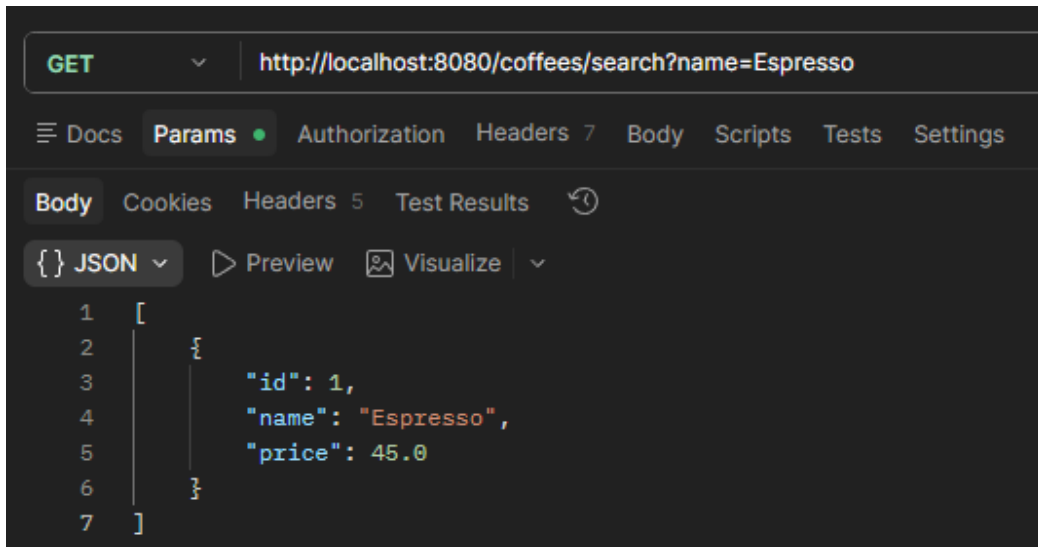
    for (Coffee coffee : coffees) {
        if (coffee.getName().toLowerCase()
            .contains(name.toLowerCase())) {
            result.add(coffee);
        }
    }

    return result;
}
```

Step 2 : เพิ่ม Code ใน CoffeeController.java

```
// Ex Bonus
@GetMapping("/search")
public List<Coffee> searchCoffee(@RequestParam String name) {
    return service.searchByName(name);
}
```

Step 3 : ทดสอบ



The screenshot shows a web browser interface for testing an API. The top bar indicates a GET request to the URL `http://localhost:8080/coffees/search?name=Espresso`. Below the URL bar, there are tabs for 'Docs', 'Params', 'Authorization', 'Headers', 'Body', 'Scripts', 'Tests', and 'Settings'. The 'Body' tab is selected, and it shows a JSON response. The JSON is displayed in a code editor with line numbers 1 through 7. The response is an array containing one object with the following properties: `"id": 1`, `"name": "Espresso"`, and `"price": 45.0`.

```
1  [
2    {
3      "id": 1,
4      "name": "Espresso",
5      "price": 45.0
6    }
7  ]
```